CGL
COMPUTER GRAPHICS LABORATORY

RISC-V 디스어셈블러 제작

20234006 최준용

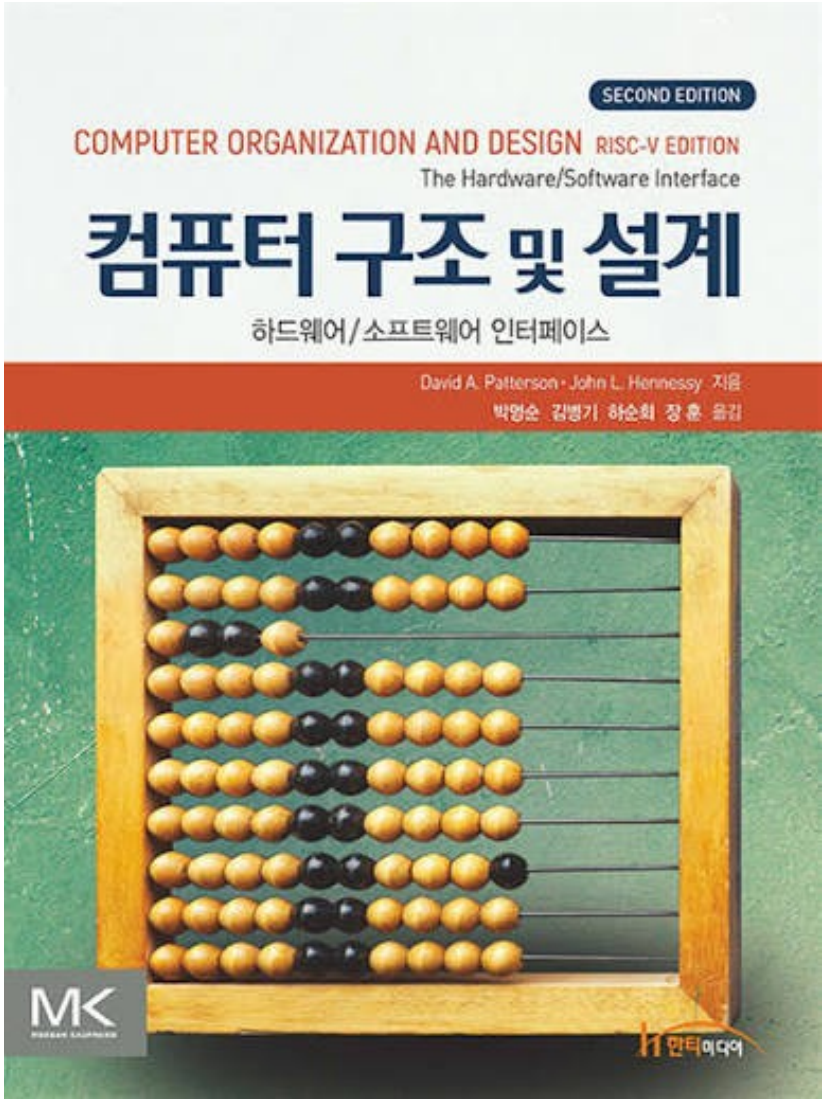


목차

CGL
COMPUTER GRAPHICS LABORATORY

- 제작 동기
- 디스어셈블러 설계 및 제작
  - decoder/rv32i.h
  - src/rv32i.cc
- 테스트 코드 작성
  - 테스트용 Parser
  - 최종 테스트 코드 (googletest)
- 디스어셈블러 테스트 및 검증
- 참고 자료

제작 동기(1)



RISC-V

오픈 소스 ISA

"Simple is the best"

x86, ARM 과 함께
아키텍처 3대장

CGL
COMPUTER GRAPHICS LABORATORY

32-bit RISC-V Instruction Formats

Instruction Formats	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Register/register	funct7							rs2					rs1					funct3			rd					opcode						
Immediate	imm[11:0]												rs1					funct3			rd					opcode						
Upper Immediate	imm[31:12]																				rd					opcode						
Store	imm[11:5]							rs2					rs1					funct3			imm[4:0]					opcode						
Branch	[12]	imm[10:5]						rs2					rs1					funct3			imm[4:1]				[11]	opcode						
Jump	[20]	imm[10:1]										[11]	imm[19:12]								rd					opcode						

모든 명령어가 4바이트
(C Extension은 2바이트)

\+ 통일성있는 명령어 형식

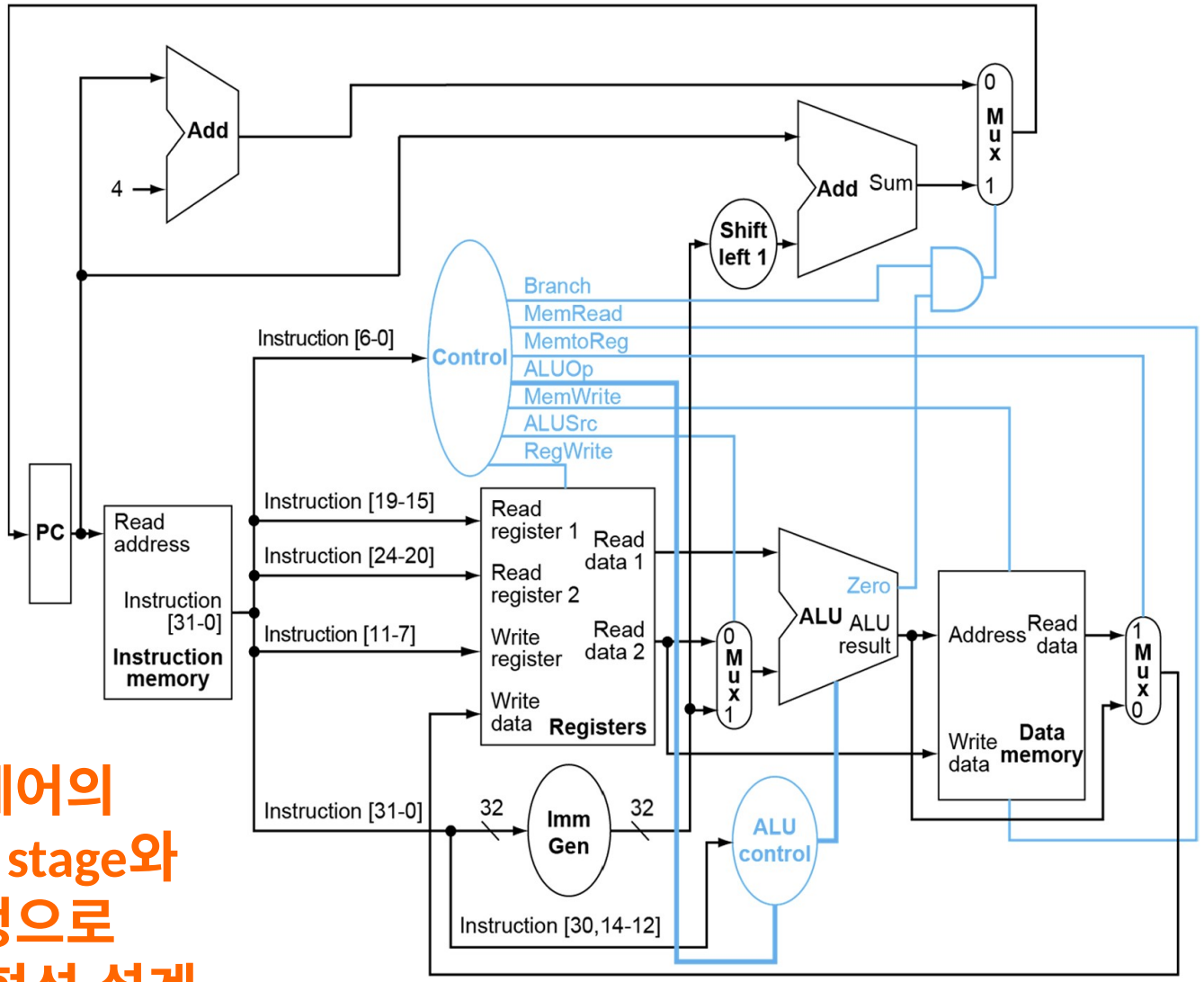
= 디스어셈블러는 만들어볼 수 있겠는걸...?

CGL
COMPUTER GRAPHICS LABORATORY

디스어셈블러 설계 및 제작

decoder/rv32i.h + rv32i.cc + parser/TestParser.h

RV32I Single Cycle Datapath



실제 하드웨어의
IF stage, ID stage와
유사한 과정으로
상속 및 다형성 설계

decoder/rv32i.h (디코더 라이브러리 API 역할)

RV32I.h

```cpp
#ifndef RV32I_H
#define RV32I_H

#include <cstdint>
#include <string>
#include <memory>

// 인스트럭션 최상위 추상 클래스
class Instruction {
protected:
  uint32_t raw_bits;
public:
  Instruction(uint32_t bits) : raw_bits(bits) {}
  virtual ~Instruction() = default;
  [[nodiscard]]
  virtual std::string disassemble([[maybe_unused]] uint32_t pc) const = 0;
};

// 형식별 인스트럭션 객체 생성 팩토리
class InstructionFactory {
public:
  static auto create(uint32_t bits) -> std::unique_ptr<Instruction>;
private:
  static auto createRType(uint32_t bits) -> std::unique_ptr<Instruction>;
  static auto createITypeLoad(uint32_t bits) -> std::unique_ptr<Instruction>;
  static auto createITypeArith(uint32_t bits) -> std::unique_ptr<Instruction>;
  static auto createITypeJump(uint32_t bits) -> std::unique_ptr<Instruction>;
  static auto createSType(uint32_t bits) -> std::unique_ptr<Instruction>;
  static auto createBType(uint32_t bits) -> std::unique_ptr<Instruction>;
  static auto createJType(uint32_t bits) -> std::unique_ptr<Instruction>;
  static auto createUType(uint32_t bits) -> std::unique_ptr<Instruction>;
 };

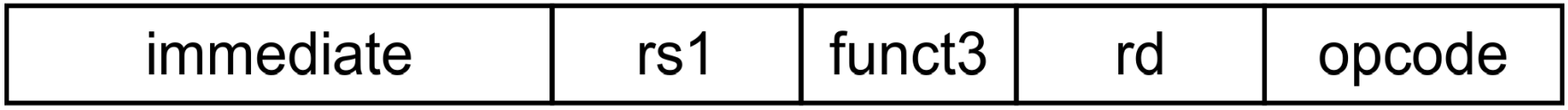
#endif
```

CGL
COMPUTER GRAPHICS LABORATORY

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

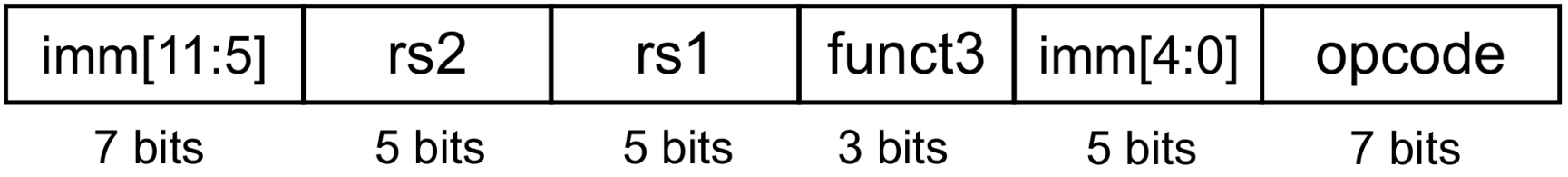
```
 9 // R-Type (rd, rs1, rs2)
10 class RInstruction : public Instruction {
11 protected:
12   std::string mnemonic;
13   uint32_t rd, rs1, rs2;
14 public:
15   RInstruction(uint32_t bits, std::string mne)
16     : Instruction(bits), mnemonic(std::move(mne)) {
17     rd      = (bits >> 7) & 0x1F;
18     rs1     = (bits >> 15) & 0x1F;
19     rs2     = (bits >> 20) & 0x1F;
20   }
21   [[nodiscard]]
22   std::string disassemble([[maybe_unused]] uint32_t pc) const override {
23     return std::format("{:<7} x{}, x{}, x{}", mnemonic, rd, rs1, rs2);
24   }
25 };
26
```

src/rv32i.cc - I-Type (산술/논리)



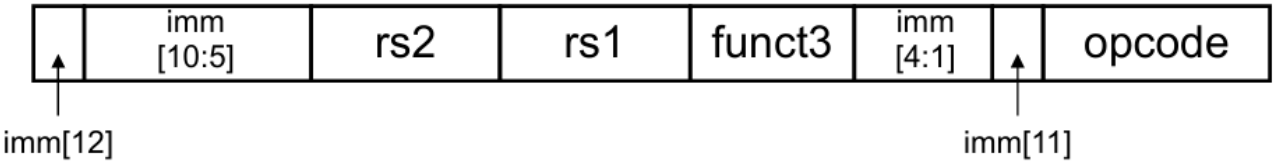

```cpp
// I-Type 산술/논리 (rd, rs1, imm)
class ArithIInstruction : public Instruction {
protected:
  std::string mnemonic;
  uint32_t rd, rs1;
  int32_t imm;
public:
  ArithIInstruction(uint32_t bits, std::string mne)
    : Instruction(bits), mnemonic(std::move(mne)) {
    rd  = (bits >> 7) & 0x1F;
    rs1 = (bits >> 15) & 0x1F;
    imm = static_cast<int32_t>(bits) >> 20;
  }
  [[nodiscard]]
  std::string disassemble([[maybe_unused]] uint32_t pc) const override {
    return std::format("{:<7} x{}, x{}, {}", mnemonic, rd, rs1, imm);
  }
};
```

src/rv32i.cc - S-Type




```cpp
// S-Type (rs2, imm(rs1))
class SInstruction : public Instruction {
protected:
  std::string mnemonic;
  uint32_t rs1, rs2;
  int32_t imm;
public:
  SInstruction(uint32_t bits, std::string mne)
    : Instruction(bits), mnemonic(std::move(mne)) {
    rs1 = (bits >> 15) & 0x1F;
    rs2 = (bits >> 20) & 0x1F;
    uint32_t imm11_5 = (bits >> 25) & 0x7F;
    uint32_t imm4_0  = (bits >> 7)  & 0x1F;
    uint32_t raw_imm = (imm11_5 << 5) | imm4_0;
    imm = static_cast<int32_t>(raw_imm << 20) >> 20;
  }
  [[nodiscard]]
  std::string disassemble([[maybe_unused]] uint32_t pc) const override {
    return std::format("{:<7} x{}, {}(x{})", mnemonic, rs2, imm, rs1);
  }
};
```

src/rv32i.cc - B-Type




```cpp
// B-Type (rs1, rs2, pc + imm)
class BInstruction : public Instruction {
protected:
  std::string mnemonic;
  uint32_t rs1, rs2;
  int32_t imm;
public:
  BInstruction(uint32_t bits, std::string mne)
    : Instruction(bits), mnemonic(std::move(mne)) {
    rs1 = (bits >> 15) & 0x1F;
    rs2 = (bits >> 20) & 0x1F;
    uint32_t imm12   = (bits >> 31) & 0x01;
    uint32_t imm11   = (bits >> 7)  & 0x01;
    uint32_t imm10_5 = (bits >> 25) & 0x3F;
    uint32_t imm4_1  = (bits >> 8)  & 0x0F;
    uint32_t raw_imm = (imm12 << 12)
                     | (imm11 << 11)
                     | (imm10_5 << 5)
                     | (imm4_1 << 1);
    imm = static_cast<int32_t>(raw_imm << 19) >> 19;
  }
  [[nodiscard]]
  std::string disassemble(uint32_t pc) const override {
    auto target = static_cast<uint32_t>(static_cast<int32_t>(pc) + imm);
    return std::format("{:<7} x{}, x{}, 0x{:08x}", mnemonic, rs1, rs2, target);
  }
};
```

src/rv32i.cc - 팩토리 구현 (1)

CGL
COMPUTER GRAPHICS LABORATORY

```cpp
// Opcode 열거형
enum class Opcode : uint8_t {
  RType = 0x33,
  ITypeArith = 0x13,
  ITypeLoad = 0x03,
  ITypeJump = 0x67,
  SType = 0x23,
  BType = 0x63,
  UTypeLoad = 0x37,
  UTypeAddi = 0x17,
  JType = 0x6F
};
```

Opcode로 1차 분기(ID stage)

```cpp
auto InstructionFactory::create(uint32_t bits)
-> std::unique_ptr<Instruction> {
  auto opcode = static_cast<Opcode>(bits & 0x7F);
  switch (opcode) {
    case Opcode::RType: return createRType(bits);
    case Opcode::ITypeLoad: return createITypeLoad(bits);
    case Opcode::ITypeArith: return createITypeArith(bits);
    case Opcode::ITypeJump: return createITypeJump(bits);
    case Opcode::SType: return createSType(bits);
    case Opcode::BType: return createBType(bits);
    case Opcode::JType: return createJType(bits);
    case Opcode::UTypeLoad:
    case Opcode::UTypeAddi: return createUType(bits);
    default: break;
  }
  return std::make_unique<UnknownInstr>(bits);
}
```

funct3, funct7 필드로 2차 분기(ID stage)

```
226
227 auto InstructionFactory::createRType(uint32_t bits)
228 -> std::unique_ptr<Instruction> {
229   uint32_t funct3 = (bits >> 12) & 0x07;
230   uint32_t funct7 = (bits >> 25) & 0x7F;
231   switch (funct3) {
232     case 0x0:
233       if (funct7 == 0x00) { return std::make_unique<RInstruction>(bits, "add"); }
234       if (funct7 == 0x20) { return std::make_unique<RInstruction>(bits, "sub"); }
235       break;
236     case 0x1: return std::make_unique<RInstruction>(bits, "sll");
237     case 0x2: return std::make_unique<RInstruction>(bits, "slt");
238     case 0x3: return std::make_unique<RInstruction>(bits, "sltu");
239     case 0x4: return std::make_unique<RInstruction>(bits, "xor");
240     case 0x5:
241       if (funct7 == 0x00) { return std::make_unique<RInstruction>(bits, "srl"); }
242       if (funct7 == 0x20) { return std::make_unique<RInstruction>(bits, "sra"); }
243       break;
244     case 0x6: return std::make_unique<RInstruction>(bits, "or");
245     case 0x7: return std::make_unique<RInstruction>(bits, "and");
246     default: break;
247   }
248   return std::make_unique<UnknownInstr>(bits);
249 }
250
```

CGL
COMPUTER GRAPHICS LABORATORY

테스트 코드 작성

parser/TestParser.h + tests/Test_disasm-rv32i.cc

테스트용 파서 - parser/TestParser.h (1)

CGL
COMPUTER GRAPHICS LABORATORY

TestParser.h+

```cpp
#ifndef TEST_PARSER_H
#define TEST_PARSER_H

#include <cstdint>
#include <cstring>
#include <filesystem>
#include <fstream>
#include <ios>
#include <stdexcept>
#include <string>
#include <vector>

namespace Parser {

inline std::vector<uint32_t>
parseBinary(const std::filesystem::path &filepath) {
  if (!std::filesystem::exists(filepath)) {
    throw std::runtime_error("파일이 존재하지 않습니다: " + filepath.string());
  }

  std::ifstream file(filepath, std::ios::binary | std::ios::ate);
  if (!file.is_open()) {
    throw std::runtime_error("파일 열기 실패: " + filepath.string());
  }

  std::streamsize raw_size = file.tellg();
  if (raw_size < 0) {
    throw std::runtime_error("파일 크기 읽어오기 실패.");
  }

  auto file_size = static_cast<size_t>(raw_size);
  if (file_size == 0) {
    throw std::runtime_error("파일이 비어있습니다 (0 bytes)");
  }
  if (file_size % sizeof(uint16_t) != 0) {
    throw std::runtime_error(
        "잘못된 바이너리 파일 크기 (2바이트 배수가 아님): " +
        std::to_string(file_size) + " bytes");
  }
```

본격적인 파싱 전,
파일 유효 검사 및
예외 처리

테스트용 파서 - parser/TestParser.h (2)

CGL
COMPUTER GRAPHICS LABORATORY

전체 바이너리를 1바이트 단위 버퍼(burrer)에 통째로 옮겨 담아 두기

16비트 압축 명령어 여부 확인하고, 그에 따라 바이너리를 명령어 단위로 쪼개서 명령어 단위 버퍼 (instructions)에 저장

ID stage

```
40
41    file.seekg(0, std::ios::beg);
42    std::vector<uint8_t> buffer(file_size);
43    if (!file.read(reinterpret_cast<char*>(buffer.data()),
44                   static_cast<std::streamsize>(file_size))) {
45      throw std::runtime_error("파일 읽기 I/O 에러");
46    }
47
48    std::vector<uint32_t> instructions;
49    size_t pos = 0;
50    while (pos < file_size) {
51      if (pos + sizeof(uint16_t) > file_size) {
52        throw std::runtime_error("파일이 비정상적으로 잘렸습니다.");
53      }
54      uint16_t half = 0;
55      std::memcpy(&half, &buffer[pos], sizeof(uint16_t));
56      if ((half & 0x3) != 0x3) {
57        // 16비트 압축 명령어
58        instructions.push_back(static_cast<uint32_t>(half));
59        pos += 2;
60      } else {
61        // 32비트 표준 명령어
62        if (pos + sizeof(uint32_t) > file_size) {
63          throw std::runtime_error("32비트 명령어 크기가 부족합니다.");
64        }
65        uint32_t full = 0;
66        std::memcpy(&full, &buffer[pos], sizeof(uint32_t));
67        instructions.push_back(full);
68        pos += 4;
69      }
70    }
71    return instructions;
72  }
73
74  }
75
76  #endif
```

최종 테스트 코드 (googletest)

CGL
COMPUTER GRAPHICS LABORATORY

Test_disasm-rv32i.cc+

```cpp
#include "disasm/RV32I.h"
#include "disasm/TestParser.h"
#include "gtest/gtest.h"

#include <cstdint>
#include <filesystem>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

TEST(DisasmIntegrationTest, DumpTestBinary) {
  uint32_t pc = 0x00000000;
  std::string file_path = "/home/junyong/cpp-lab/resource/test.bin";

  ASSERT_TRUE(std::filesystem::exists(file_path))
      << "테스트 바이너리가 존재하지 않습니다.";

  auto instructions = Parser::parseBinary(file_path);
  ASSERT_FALSE(instructions.empty()) << "파싱한 명령어가 공백상태입니다.";

  std::cout << "\n디스어셈블리 결과 검증 테스트\n";
  std::cout << "디스어셈블 대상 파일: " << file_path << '\n';
  std::cout << "-------------------------------------------------------\n";
  std::cout << " PC 주소       바이너리 코드       디스어셈블 결과\n";
  std::cout << "-------------------------------------------------------\n";

  for (std::uint32_t bits : instructions) {
    auto instr = InstructionFactory::create(bits);
    std::cout << "0x" << std::setw(8) << std::setfill('0') << std::hex << pc
              << "    "
              << "0x" << std::setw(8) << std::setfill('0') << std::hex << bits
              << "    " << instr->disassemble(pc) << "\n";
    pc += (bits & 0x3) == 0x3 ? 4 : 2;
  }

  SUCCEED();
}
```

PC값 업데이트에도
16비트 압축 명령어(C Extension)
여부 반영

테스트용 C파일 (test.c)

CGL
COMPUTER GRAPHICS LABORATORY

```
junyong@fedora ~/cpp-lab
main ❯ bat resource/test.c
```

File: resource/test.c

```c
int sum(int arr[], int n) {
  int total = 0;
  int* iter = arr;
  int* end = arr + n;
  for (; iter != end; ++iter) {
    total += *iter;
  }
  return total;
}

int main(void) {
  int array[] = {1, 2, 3, 4, 5};
  int size = sizeof(array) / sizeof(array[0]);
  int result = sum(array, size);
  return 0;
}
```

CGL
COMPUTER GRAPHICS LABORATORY

디스어셈블러 테스트 및 검증

riscv32 크로스 컴파일 및 .bin 파일 생성

.text 섹션만 추출한 .bin 파일 생성

```
junyong@fedora ~/cpp-lab/resource
main ! ❯ clang --config ~/.config/clang/riscv32.cfg -O1 -c test.c -o test.out
junyong@fedora ~/cpp-lab/resource
main ! ❯ llvm-objcopy -O binary --only-section=.text test.out test.bin
junyong@fedora ~/cpp-lab/resource
main ! ❯ ls -l
total 24
-rw-r--r-- 1 junyong junyong  233 Aug 26 23:10 map_file.txt
-rw-r--r-- 1 junyong junyong  347 Sep  5 17:08 query_file.txt
-rw-r--r-- 1 junyong junyong   58 Nov  2 01:03 test.bin
-rw-r--r-- 1 junyong junyong  303 Nov  2 00:12 test.c
-rw-r--r-- 1 junyong junyong 1132 Nov  2 01:03 test.out
-rw-r--r-- 1 junyong junyong   91 Aug 26 23:11 to_transform.txt
```

테스트 결과 (test.bin 디스어셈블)

```
┌─ junyong@fedora ~/cpp-lab
└─ main ❯ ./build/asan/tests/disasm-rv32i_test
Running main() from /home/junyong/cpp-lab/build/asan/_deps/googl
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from DisasmIntegrationTest
[ RUN      ] DisasmIntegrationTest.DumpTestBinary

디스어셈블리 결과 검증 테스트
디스어셈블 대상 파일: /home/junyong/cpp-lab/resource/test.bin
-------------------------------------------------
 PC 주소        바이너리 코드        디스어셈블 결과
-------------------------------------------------
0x00000000    0x00058063    beq     x11, x0, 0x00000000
0x00000004    0x00050613    addi    x12, x10, 0
0x00000008    0x00000513    addi    x10, x0, 0
0x0000000c    0x00259593    slli    x11, x11, 2
0x00000010    0x00062683    lw      x13, 0(x12)
0x00000014    0xffc58593    addi    x11, x11, -4
0x00000018    0x00a68533    add     x10, x13, x10
0x0000001c    0x00460613    addi    x12, x12, 4
0x00000020    0xfe0598e3    bne     x11, x0, 0x00000010
0x00000024    0x00008067    jalr    x0, 0(x1)
0x00000028    0x00000513    addi    x10, x0, 0
0x0000002c    0x00008067    jalr    x0, 0(x1)
0x00000030    0x00000001    unknown
0x00000032    0x00000513    addi    x10, x0, 0
0x00000036    0x00008067    jalr    x0, 0(x1)
[       OK ] DisasmIntegrationTest.DumpTestBinary (15 ms)
[----------] 1 test from DisasmIntegrationTest (15 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (15 ms total)
[  PASSED  ] 1 test.
```

실제 디스어셈블러와 대조하며 검증

CGL
COMPUTER GRAPHICS LABORATORY

disasm-rv32i_test

```
junyong@fedora ~/cpp-lab
main ❯ ./build/asan/tests/disasm-rv32i_test
Running main() from /home/junyong/cpp-lab/build/asan/_de
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from DisasmIntegrationTest
[ RUN      ] DisasmIntegrationTest.DumpTestBinary

디스어셈블리 결과 검증 테스트
디스어셈블 대상 파일: /home/junyong/cpp-lab/resource/tes
------------------------------------------------
 PC 주소        바이너리 코드        디스어셈블 결과
------------------------------------------------
0x00000000    0x00058063    beq     x11, x0, 0x00000000
0x00000004    0x00050613    addi    x12, x10, 0
0x00000008    0x00000513    addi    x10, x0, 0
0x0000000c    0x00259593    slli    x11, x11, 2
0x00000010    0x00062683    lw      x13, 0(x12)
0x00000014    0xffc58593    addi    x11, x11, -4
0x00000018    0x00a68533    add     x10, x13, x10
0x0000001c    0x00460613    addi    x12, x12, 4
0x00000020    0xfe0598e3    bne     x11, x0, 0x00000010
0x00000024    0x00008067    jalr    x0, 0(x1)
0x00000028    0x00000513    addi    x10, x0, 0
0x0000002c    0x00008067    jalr    x0, 0(x1)
0x00000030    0x00000001    unknown
0x00000032    0x00000513    addi    x10, x0, 0
0x00000036    0x00008067    jalr    x0, 0(x1)
[       OK ] DisasmIntegrationTest.DumpTestBinary (15 ms
[----------] 1 test from DisasmIntegrationTest (15 ms to
```

llvm-objdump

```
junyong@fedora ~/cpp-lab
main ❯ llvm-objdump --triple=riscv32 \
>  -M numeric,no-aliases -d resource/test.out

resource/test.out:      file format elf32-littleriscv

Disassembly of section .text:

00000000 <sum>:
       0: 00058063      beq     x11, x0, 0x0 <sum>
       4: 00050613      addi    x12, x10, 0x0
       8: 00000513      addi    x10, x0, 0x0
       c: 00259593      slli    x11, x11, 0x2
      10: 00062683      lw      x13, 0x0(x12)
      14: ffc58593      addi    x11, x11, -0x4
      18: 00a68533      add     x10, x13, x10
      1c: 00460613      addi    x12, x12, 0x4
      20: fe0598e3      bne     x11, x0, 0x10 <sum+0x10>
      24: 00008067      jalr    x0, 0x0(x1)
      28: 00000513      addi    x10, x0, 0x0
      2c: 00008067      jalr    x0, 0x0(x1)
      30: 0001          <unknown>

00000032 <main>:
      32: 00000513      addi    x10, x0, 0x0
      36: 00008067      jalr    x0, 0x0(x1)
junyong@fedora ~/cpp-lab
main ❯
```

한계점 + 다음 목표

```
[ RUN      ] DisasmIntegrationTest.DumpTestBinary

디스어셈블리 결과 검증 테스트
디스어셈블 대상 파일: /home/junyong/cpp-lab/resource/test.out
-----------------------------------------------------
 PC 주소        바이너리 코드        디스어셈블 결과
-----------------------------------------------------
0x00000000    0x464c457f    unknown
0x00000004    0x00000101    unknown
0x00000006    0x00000001    unknown
0x00000008    0x00000000    unknown
0x0000000a    0x00000000    unknown
0x0000000c    0x00000000    unknown
0x0000000e    0x00000000    unknown
0x00000010    0x00000001    unknown
0x00000012    0x000100f3    unknown
0x00000016    0x00000000    unknown
0x00000018    0x00000000    unknown
0x0000001a    0x00000000    unknown
0x0000001c    0x00000000    unknown
0x0000001e    0x00000000    unknown
0x00000020    0x000002b4    unknown
0x00000022    0x00000000    unknown
0x00000024    0x00000004    unknown
0x00000026    0x00000000    unknown
0x00000028    0x00000034    unknown
0x0000002a    0x00000000    unknown
0x0000002c    0x00000000    unknown
0x0000002e    0x00000028    unknown
0x00000030    0x0001000b    unknown
0x00000034    0x00058063    beq     x11, x0, 0x00000034
0x00000038    0x00050613    addi    x12, x10, 0
0x0000003c    0x00000513    addi    x10, x0, 0
0x00000040    0x00259593    slli    x11, x11, 2
0x00000044    0x00062683    lw      x13, 0(x12)
0x00000048    0xffc58593    addi    x11, x11, -4
0x0000004c    0x00a68533    add     x10, x13, x10
0x00000050    0x00460613    addi    x12, x12, 4
0x00000054    0xfe0598e3    bne     x11, x0, 0x00000044
0x00000058    0x00008067    jalr    x0, 0(x1)
0x0000005c    0x00000513    addi    x10, x0, 0
0x00000060    0x00008067    jalr    x0, 0(x1)
0x00000064    0x00000001    unknown
0x00000066    0x00000513    addi    x10, x0, 0
0x0000006a    0x00008067    jalr    x0, 0(x1)
0x0000006e    0x00006300    unknown
0x00000070    0x0000616c    unknown
```

실행 파일(.out)을
파싱하면...

.text 섹션 or 실행 가능한
PT_LOAD 세그먼트만
추출해줄 **ELF Parser가 필요!**

참고 문서 및 사이트

CGL
COMPUTER GRAPHICS LABORATORY

- 컴퓨터 구조 및 설계 RISC-V Edition (David A. Patterson, John Hennessy)
- riscv.org - The RISC-V Instruction Set Manual, Volume I 문서
- C++20 Get the Details (Rainer Grimm)
- Effective Modern C++ (Scott Douglas Meyers)
- cppreference.com
- https://github.com/google/googletest

CGL
COMPUTER GRAPHICS LABORATORY

THANK YOU

최준용 | thomas19204@gmail.com